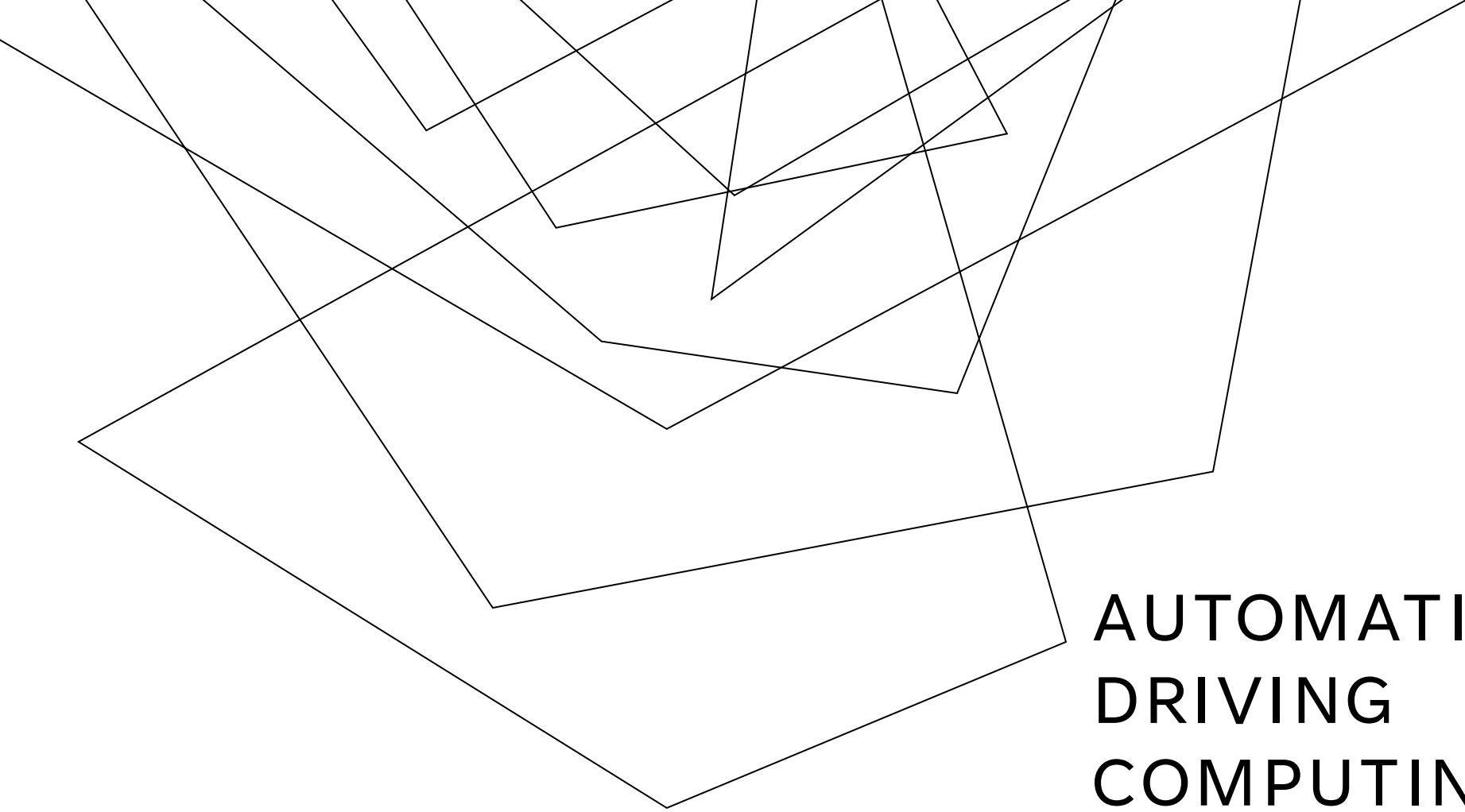




AUTOMATIC DRIVING COMPUTING

10. 3D PERCEPTION (2)





공지사항

ASSIGNMENT #1 IS OUT

- Due : 10/7 (수) 23 : 59
- 제출 기한이 지난 후에는 추가 제출 (딜레이)은 받지 않음





지난 시간 복습

3D OBJECT DETECTION (1)

1. 3D Object Detection
2. Image-based 3D Object Detection





목차

목차

1. LiDAR Data Type & Backbone
2. LiDAR-based 3D Object Detection

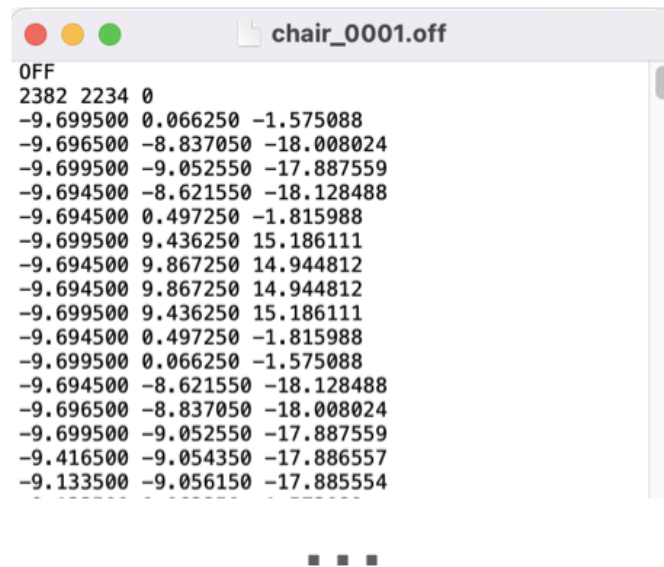


An abstract graphic design featuring two thin, dark gray lines that intersect on a light gray background. One line runs diagonally from the top-left towards the bottom-right, while the other runs from the top-right towards the bottom-left. The intersection point is located to the left of the text.

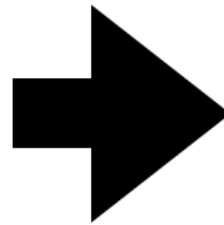
LIDAR DATA TYPE & BACKBONE

POINT CLOUD

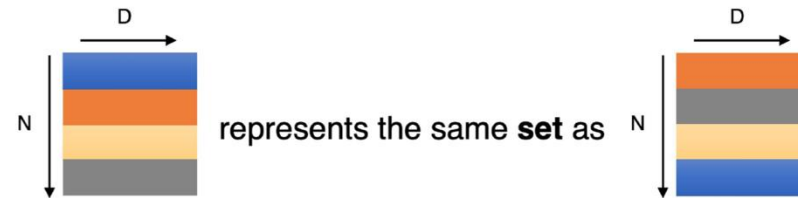
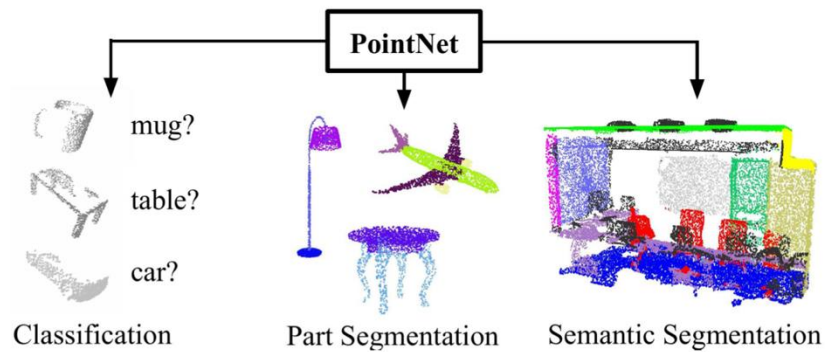
LiDAR 센서로 수집된 정보는 Point cloud 형태로 저장됨
각 point마다 (x, y, z) 좌표가 담겨 있는 [N x 3] matrix



```
OFF
2382 2234 0
-9.699500 0.066250 -1.575088
-9.696500 -8.837050 -18.008024
-9.699500 -9.052550 -17.887559
-9.694500 -8.621550 -18.128488
-9.694500 0.497250 -1.815988
-9.699500 9.436250 15.186111
-9.694500 9.867250 14.944812
-9.694500 9.867250 14.944812
-9.699500 9.436250 15.186111
-9.694500 0.497250 -1.815988
-9.699500 0.066250 -1.575088
-9.694500 -8.621550 -18.128488
-9.696500 -8.837050 -18.008024
-9.699500 -9.052550 -17.887559
-9.416500 -9.054350 -17.886557
-9.133500 -9.056150 -17.885554
...
```



LEARNING WITH POINT CLOUDS



$$f(x_1, x_2, \dots, x_n) \equiv f(x_{\pi_1}, x_{\pi_2}, \dots, x_{\pi_n}), \quad x_i \in \mathbb{R}^D$$

Examples:

$$f(x_1, x_2, \dots, x_n) = \max\{x_1, x_2, \dots, x_n\}$$

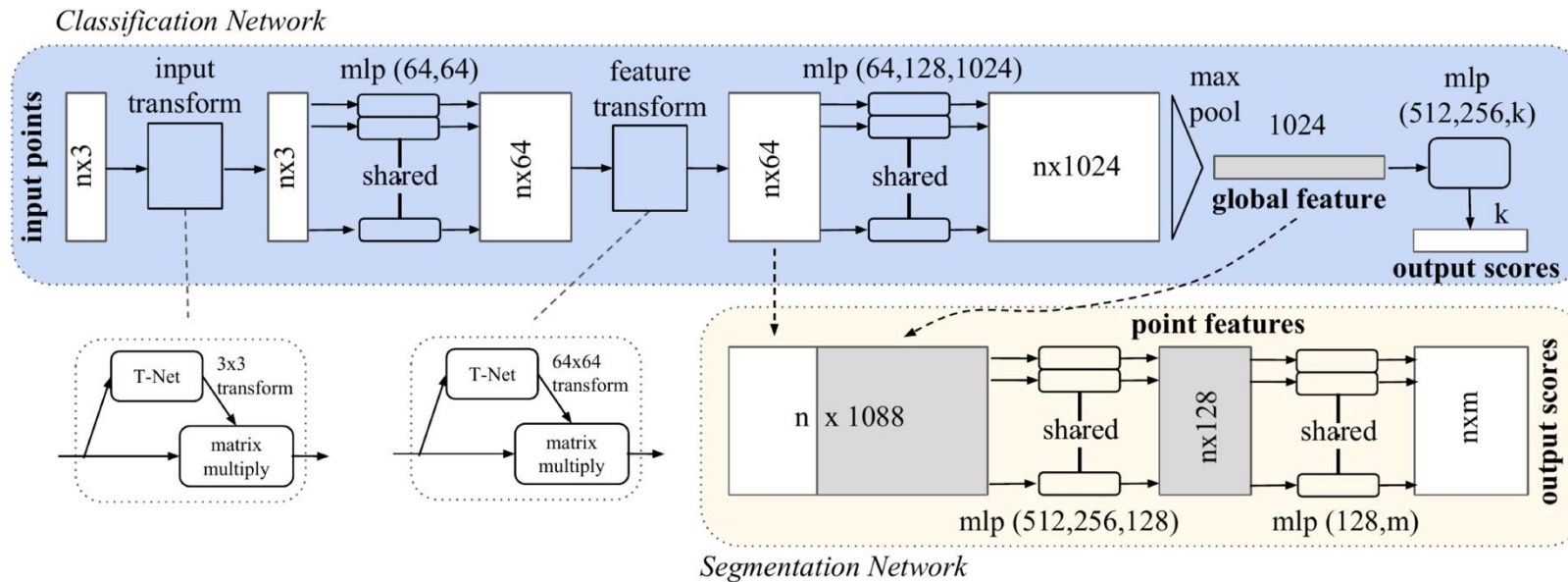
$$f(x_1, x_2, \dots, x_n) = x_1 + x_2 + \dots + x_n$$

Challenges when learning with point clouds:

- ▶ The learned model must accommodate point clouds of **arbitrary size**
- ▶ The learned model needs to be invariant to all (N!) **permutations**
- ▶ The learned model should be invariant to **rotations** of the point cloud



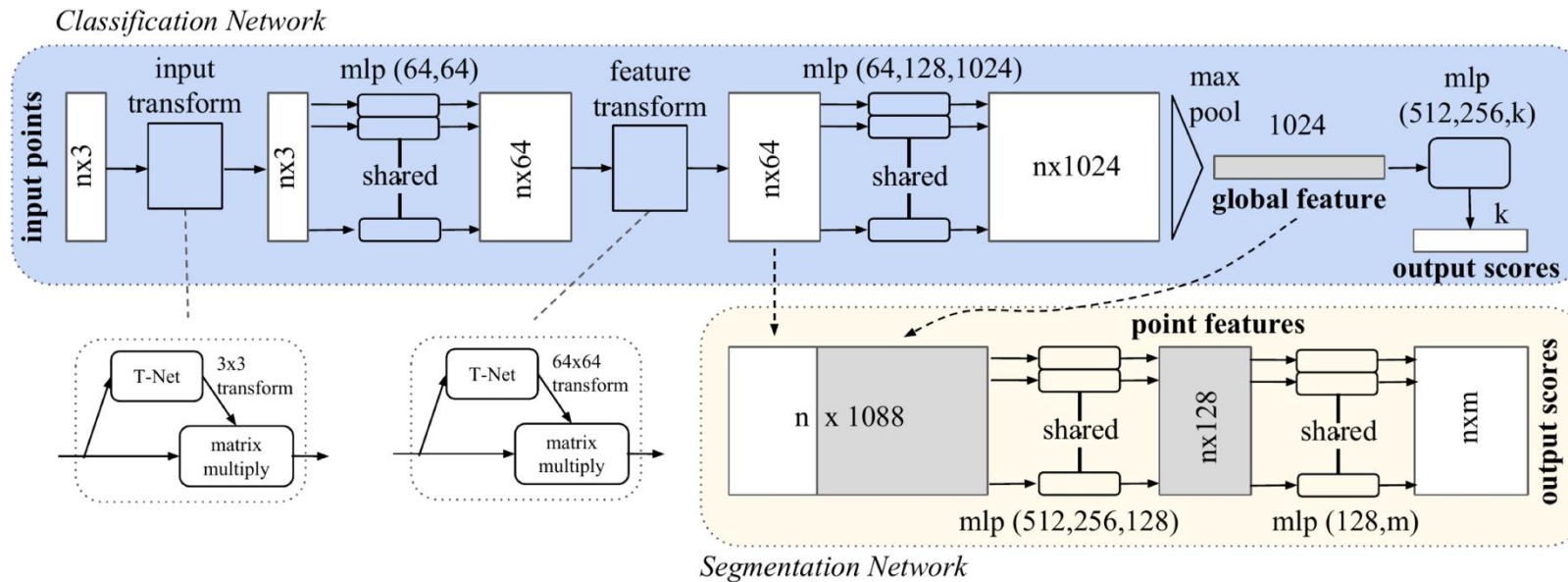
POINT CLOUD BACKBONE – POINTNET



- ▶ PointNet applies MLP per point, followed by **max-pooling** and a final (global) MLP
- ▶ Affine input/feature transformations for geometric invariance (small PointNets)

Qi, Su, Mo and Guibas: PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation. CVPR, 2017.

POINT CLOUD BACKBONE – POINTNET

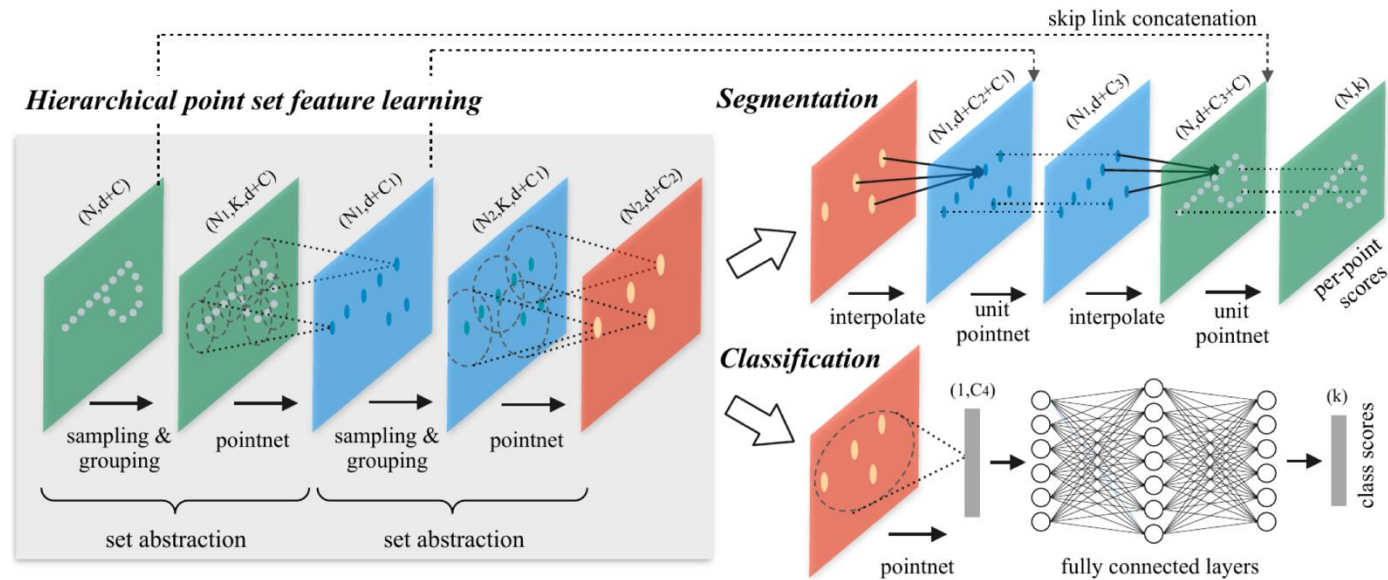


- ▶ PointNet applies MLP per point, followed by **max-pooling** and a final (global) MLP
- ▶ Affine input/feature transformations for geometric invariance (small PointNets)

Qi, Su, Mo and Guibas: PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation. CVPR, 2017.



POINT CLOUD BACKBONE – POINTNET++



- ▶ PointNet does not capture local structures and depends on global coordinates
- ▶ PointNet++ applies PointNet **recursively** on **nested partitions** of the point set
- ▶ It learns features with increasing contextual scales $\Rightarrow$ "Multi-scale PointNet"

An abstract graphic design featuring two thin, dark gray lines that intersect on a light gray background. One line runs diagonally from the top-left towards the bottom-right, while the other runs from the top-right towards the bottom-left. The intersection point is located to the left of the text.

LIDAR-BASED 3D OBJECT DETECTION

LIDAR-BASED 3D OBJECT DETECTION

- **PointRCNN: 3D Object Proposal Generation and Detection From Point Cloud (CVPR 2019)**
- **VoxelNet: End-to-End Learning for Point Cloud Based 3D Object Detection (CVPR 2018)**
- **PIXOR: Real-time 3D Object Detection from Point Clouds (CVPR 2018)**

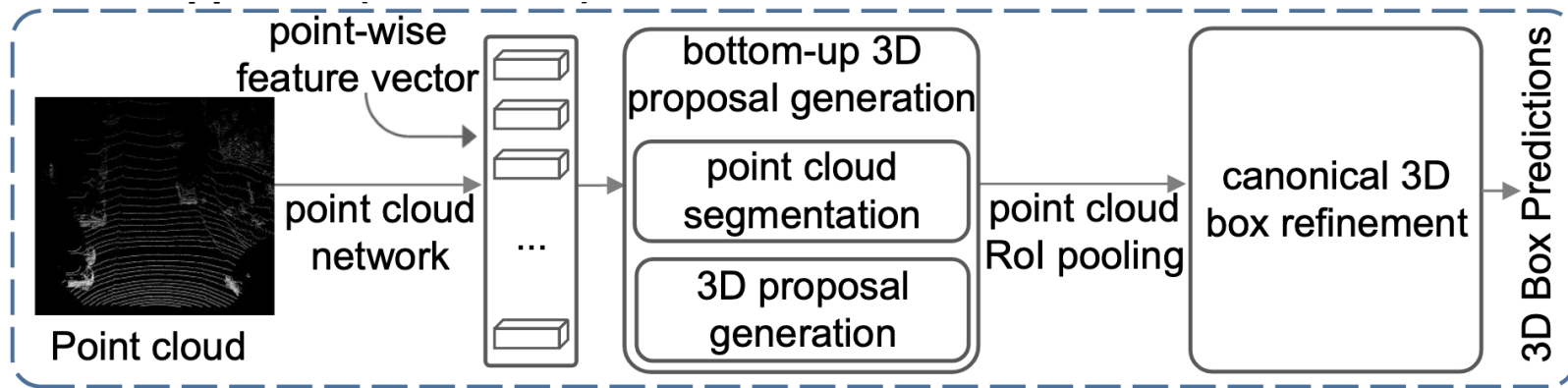


POINTRCNN

Faster R-CNN의 point cloud 버전

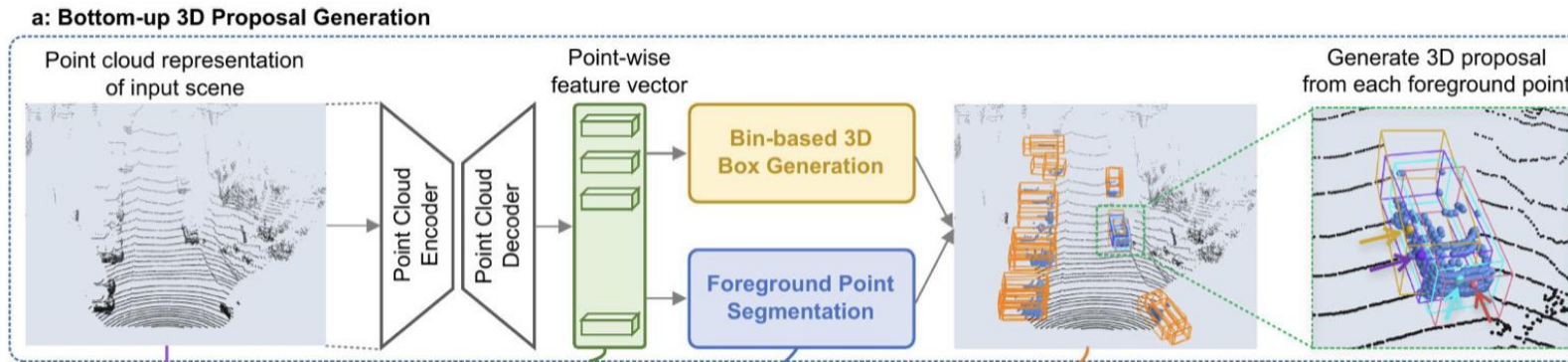
Stage 1 – Bottom-up 3D proposal generation

Stage 2 – Proposal refinement



POINTRCNN

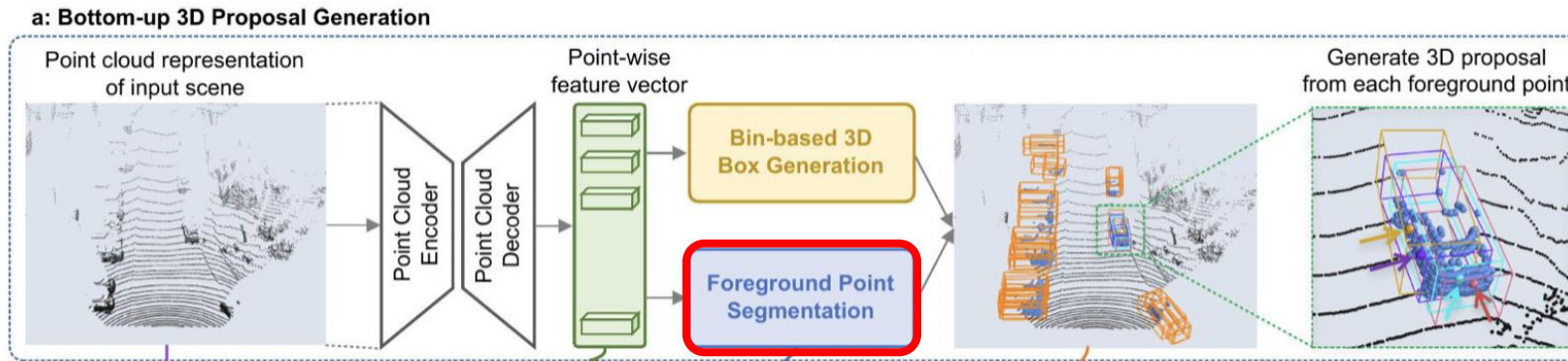
Stage 1 – Bottom-up 3D proposal generation



1. Backbone learns discriminative point-wise features with **PointNet++**
2. **3D box proposal regression** (like Faster R-CNN) and foreground segmentation (combination of regression and classification+regression (“bin-based”) losses)
3. Pass top 100 proposals to box refinement stage

POINTRCNN

Stage 1 – Bottom-up 3D proposal generation

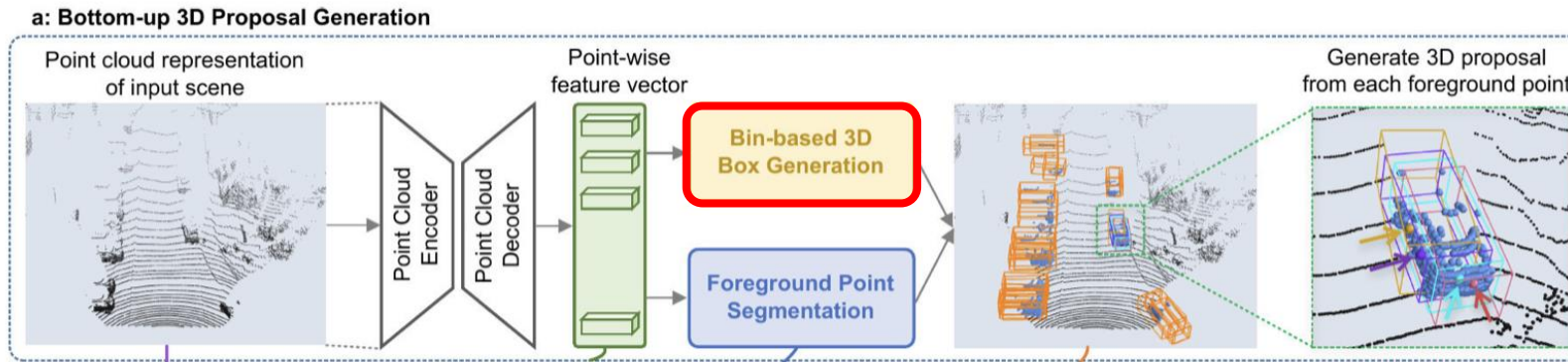


Cf) Foreground Point Segmentation :

- Ground truth 3D bounding box 안의 point들을 foreground point로 할당
- Backbone network가 contextual information을 capture하도록 하는 효과
- Foreground point로 예측된 point에 대해서만 3D proposal 생성

POINTRCNN

Stage 1 – Bottom-up 3D proposal generation

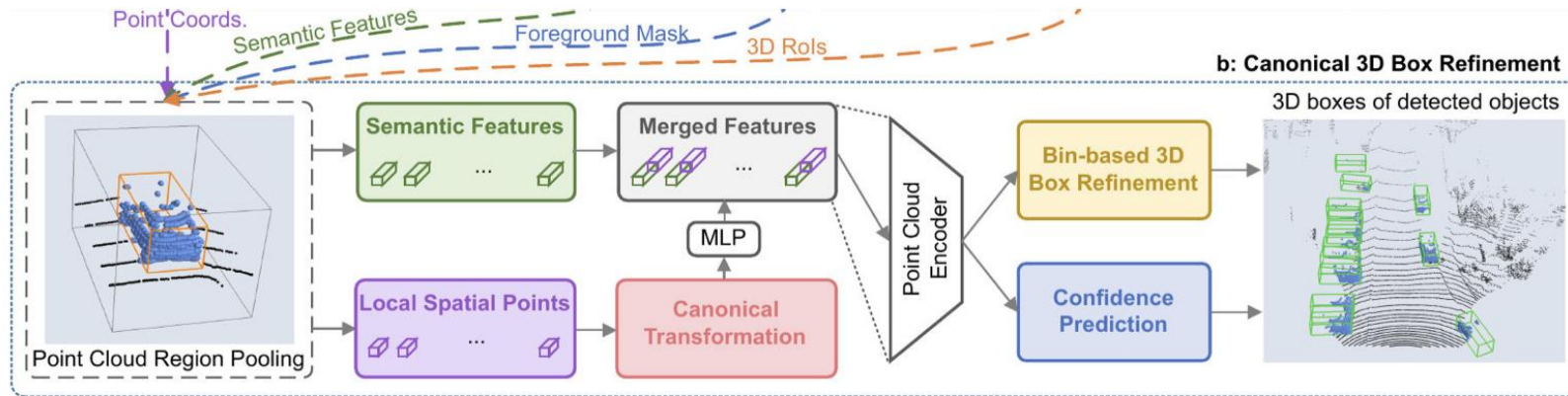


Cf) Bin-based 3D Box Generation :

- Anchor를 사용하지 않음 (Faster R-CNN와의 차이점)
- (x, y, z) - object center location : foreground point 좌표에서 regression
- (h, w, l) - object size : training set 내 클래스별 평균 사이즈에서 regression
- (θ) - object orientation : bin classification + residual regression

POINTRCNN

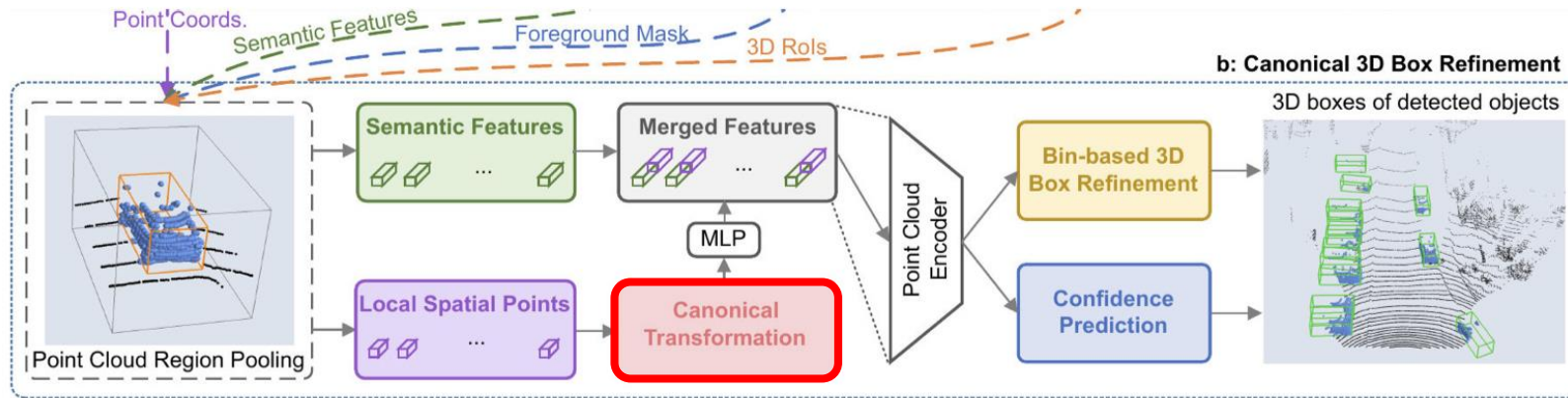
Stage 2 – Proposal refinement



1. **Pooling** of 3D point features based on 3D box proposals
2. Transform the pooled points to canonical (local) 3D box coordinates
3. **3D box refinement** by predicting residuals as in Faster R-CNN
(combination of regression and classification+regression (“bin-based”) losses)

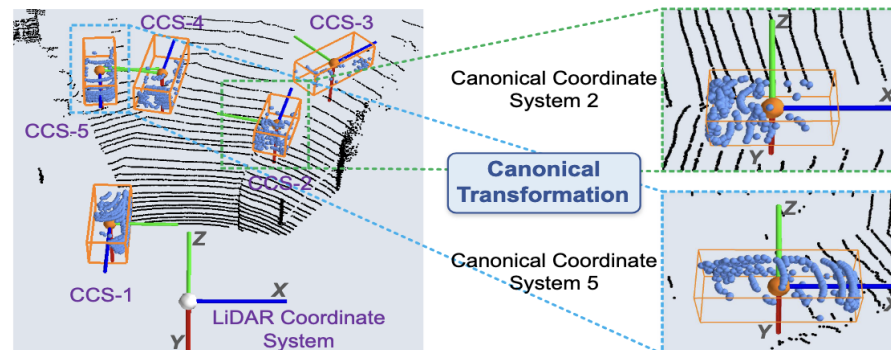
POINTRCNN

Stage 2 – Proposal refinement



Cf) Canonical Transformation :

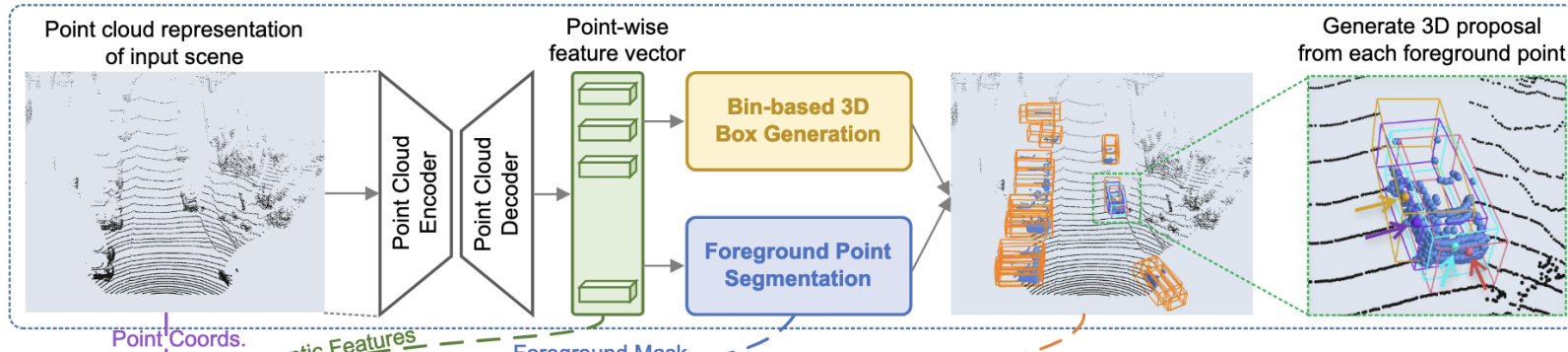
Local spatial feature learning에 용이하도록 각 proposal별로 좌표계 변환



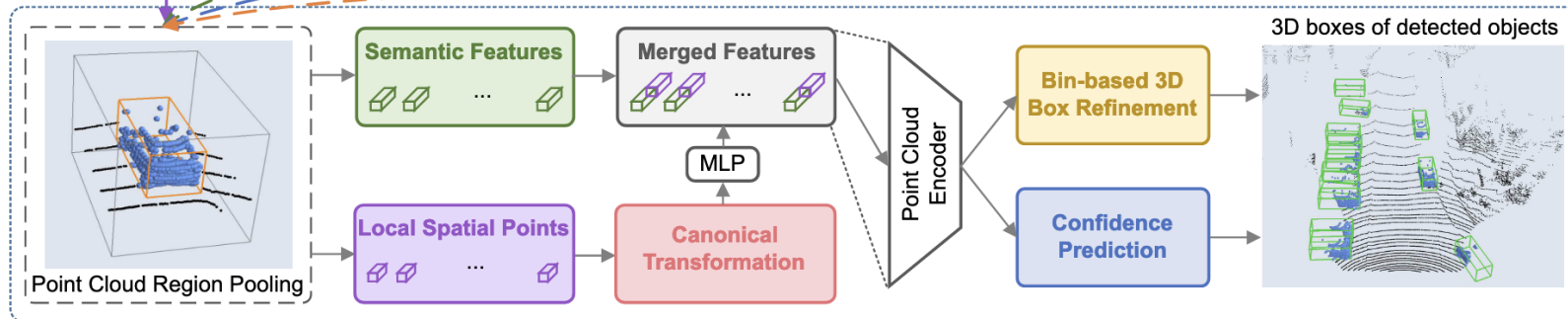
POINTRCNN

전체 pipeline

a: Bottom-up 3D Proposal Generation

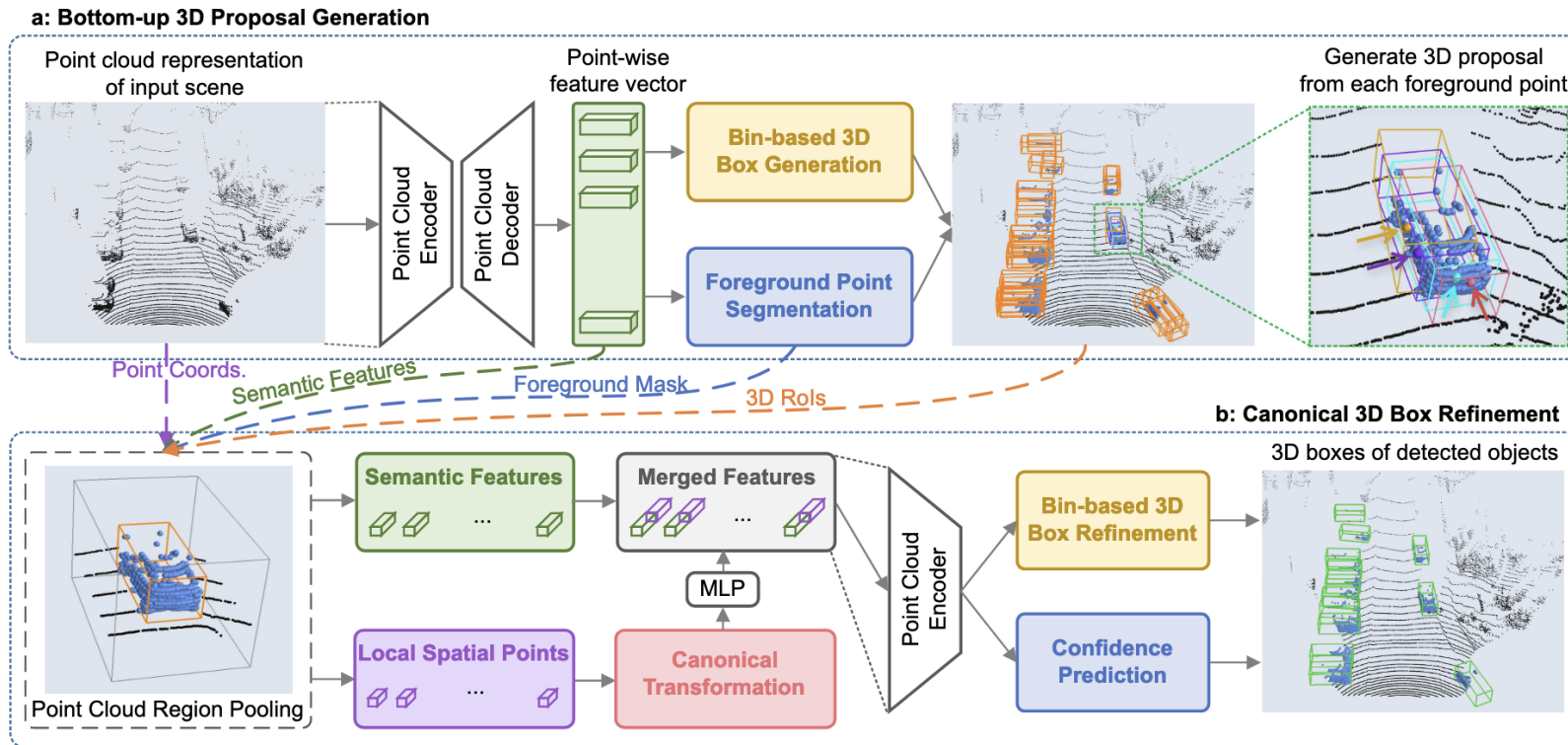


b: Canonical 3D Box Refinement



POINTRCNN

전체 pipeline

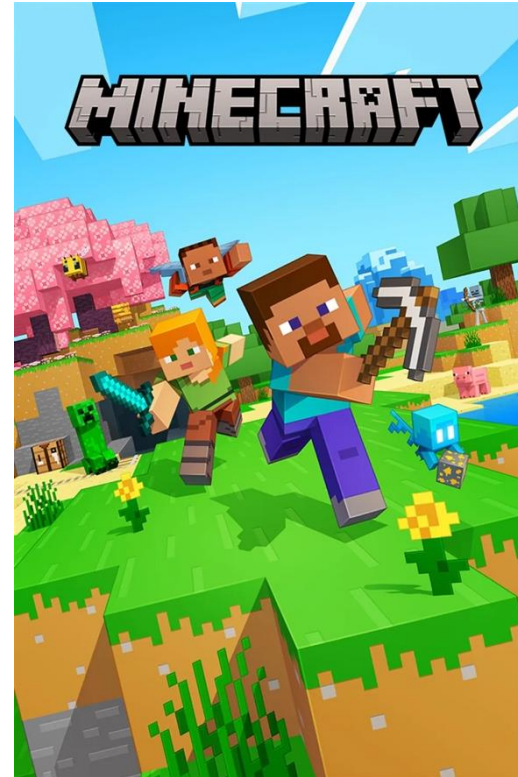
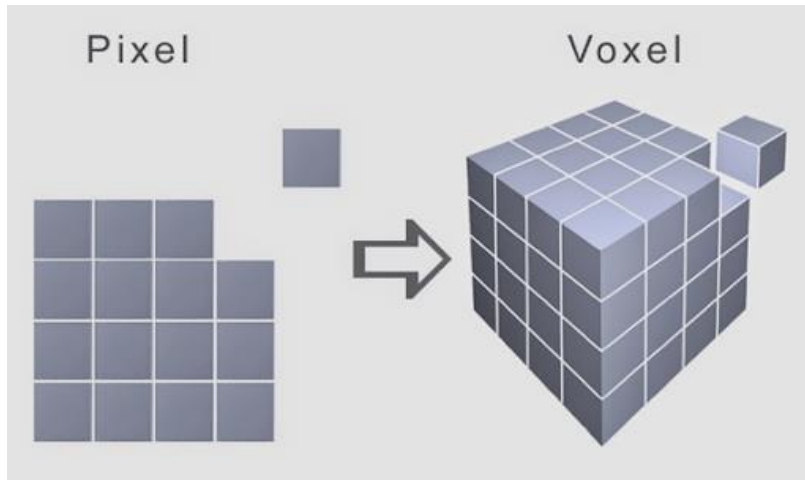


단점 : Two-stage detection pipeline → inference 느림



VOXELNET

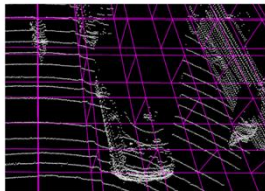
- What is “Voxel” ?
- Voxel : Volume + Pixel
 - 2차원 이미지 → Pixel들로 표현
 - 3차원 공간 → Voxel들로 표현
 - cf) 마인크래프트



VOXELNET

1) Point cloud 공간을 동일한 크기의 voxel들로 구역을 나눔 ($0.2\text{m} \times 0.2\text{m} \times 0.4\text{m}$)

Voxel
Partition

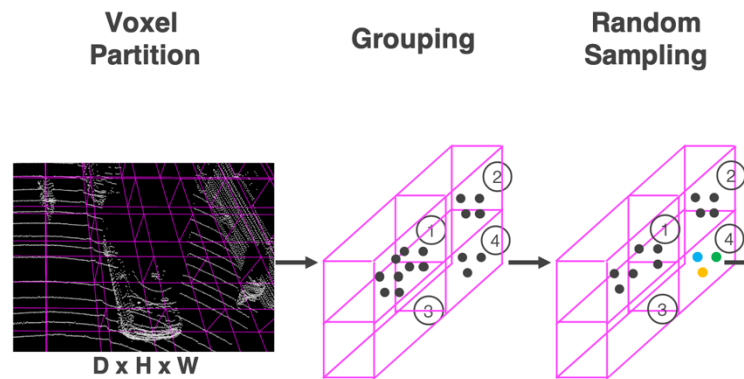


D x H x W



VOXELNET

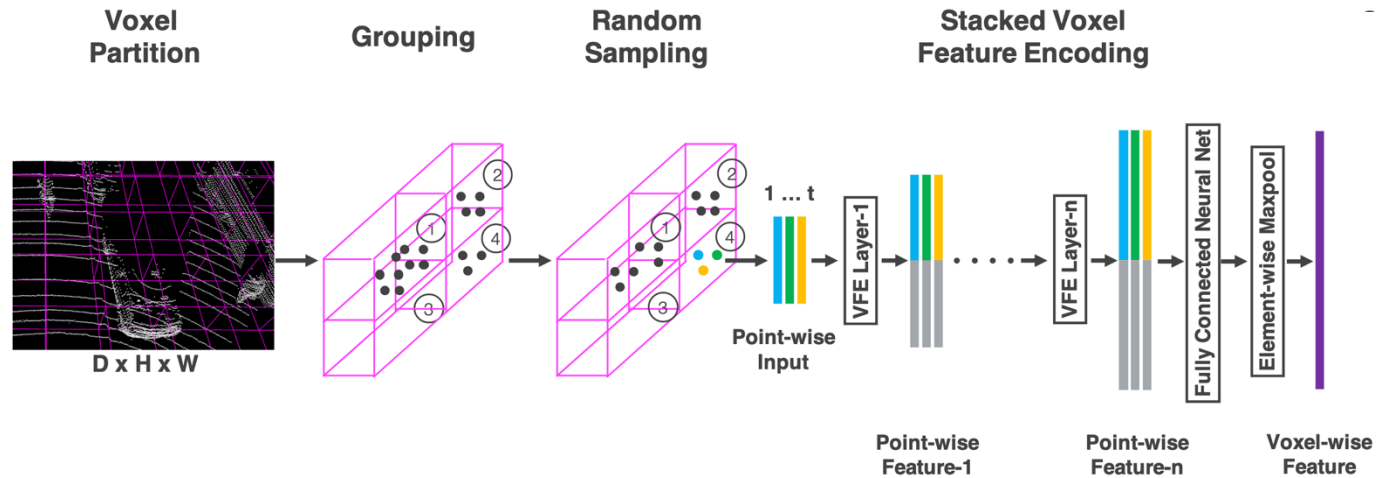
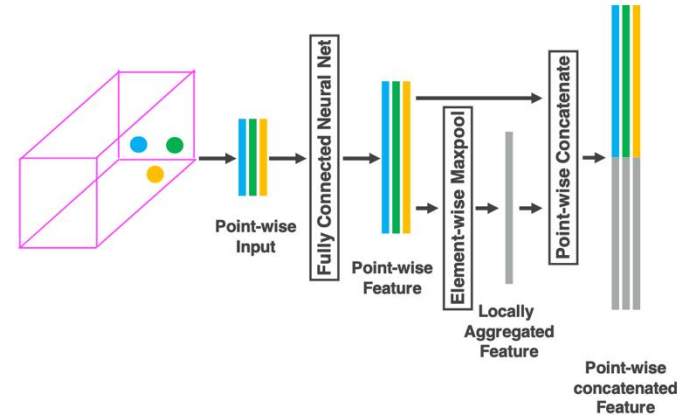
2) Voxel 내의 point 들을 group으로 묶고, group이 너무 클 경우 balance 조정



VOXELNET

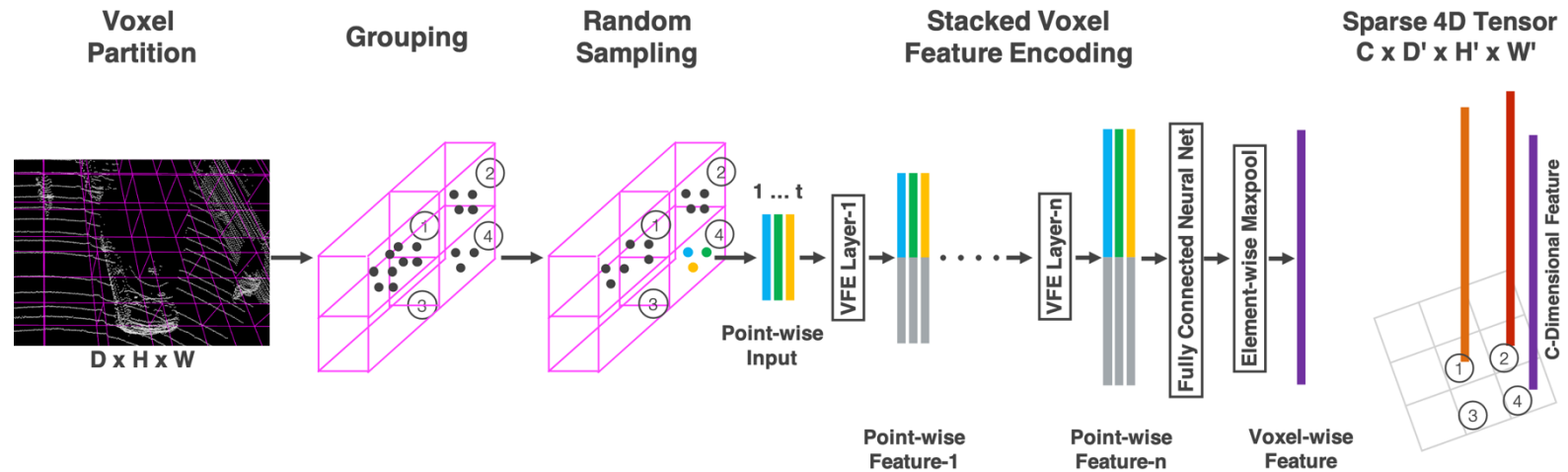
3) 각 voxel group 내에서 feature encoding 수행 (VFE layer)

Cf) VFE Layer-1 :



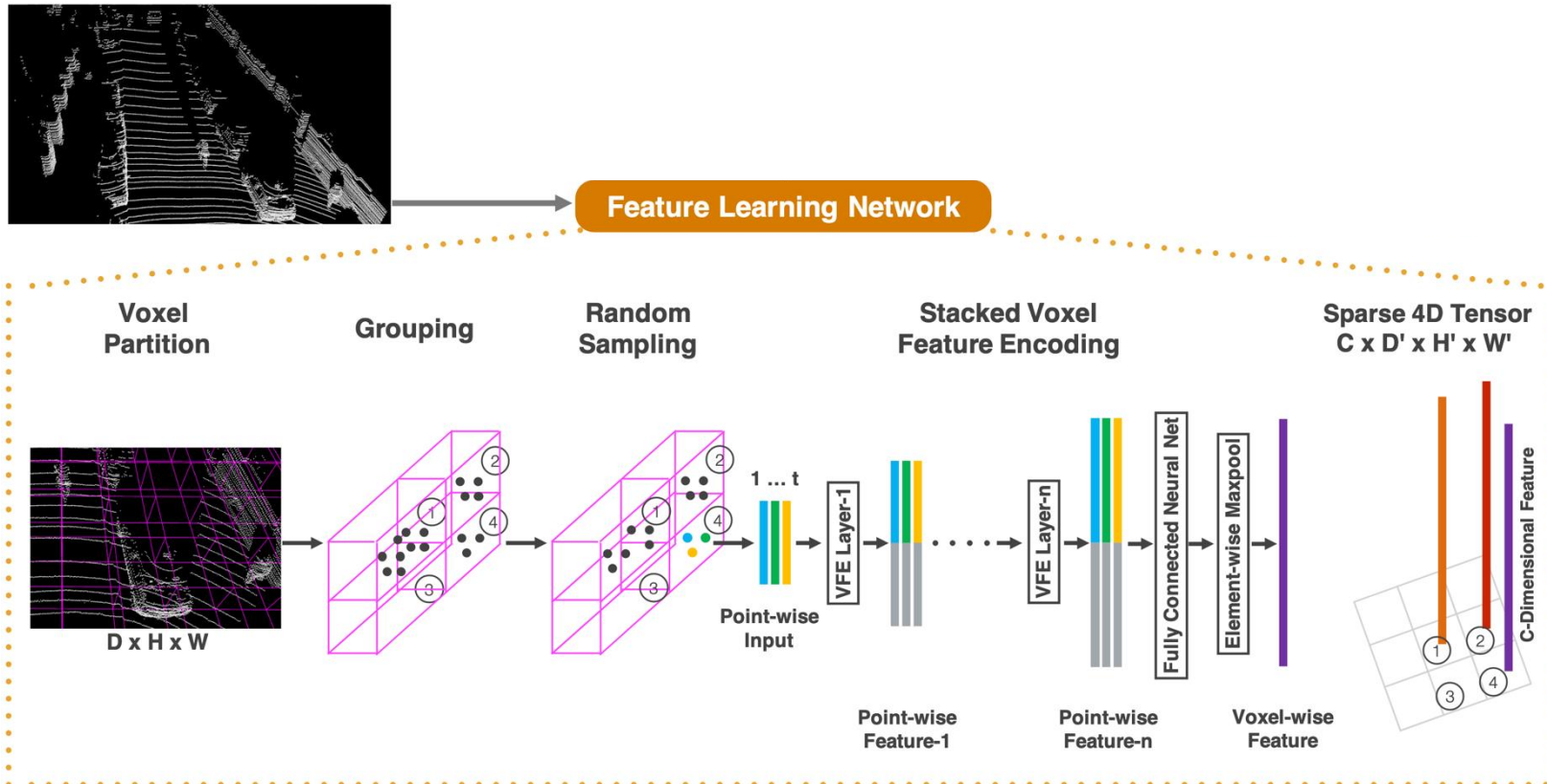
VOXELNET

4) 전체 point cloud 공간의 feature를 담은 4D Tensor 계산됨



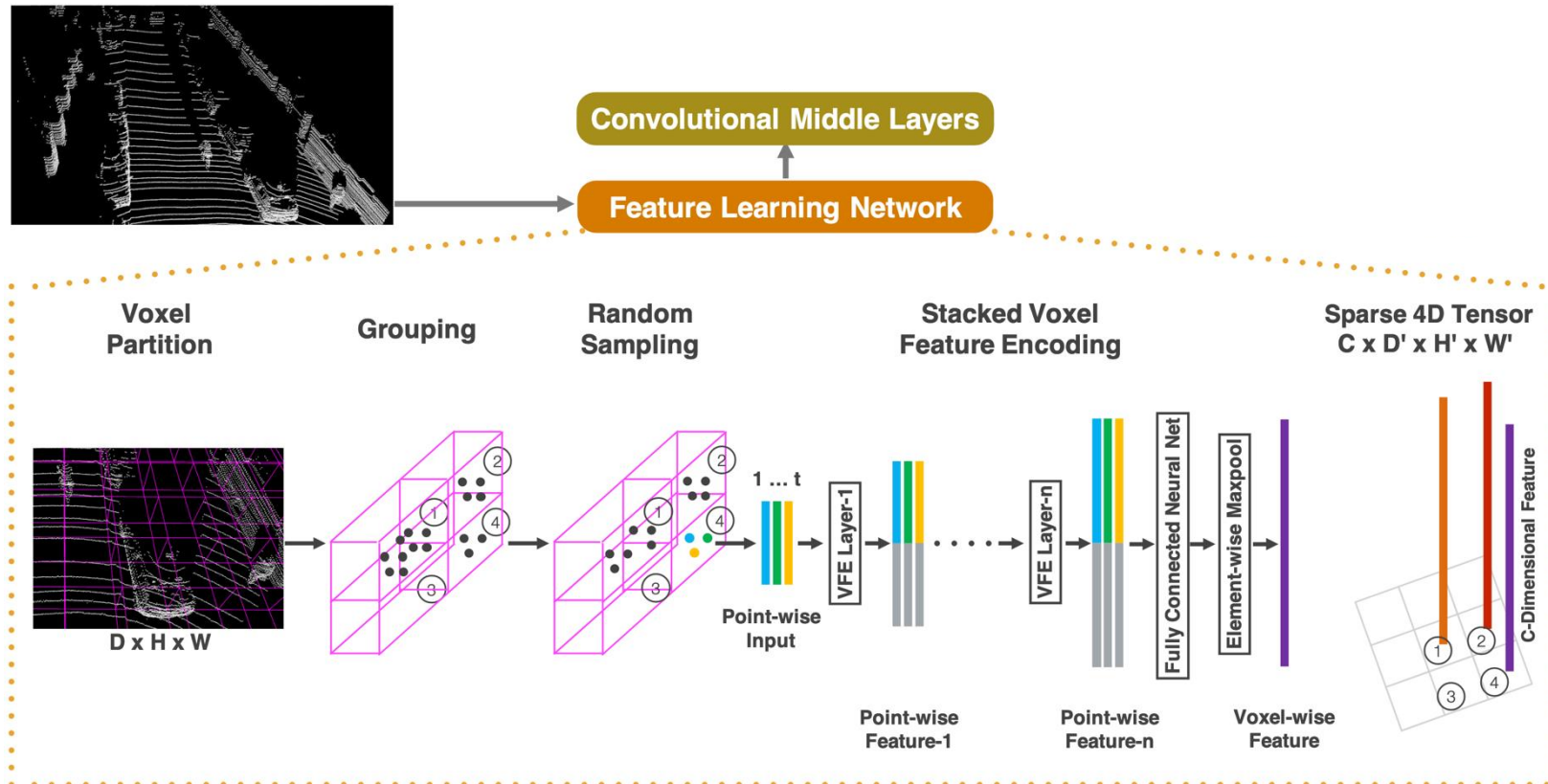
VOXELNET

4) 전체 point cloud 공간의 feature를 담은 4D Tensor 계산됨 “Feature Learning Network”



VOXELNET

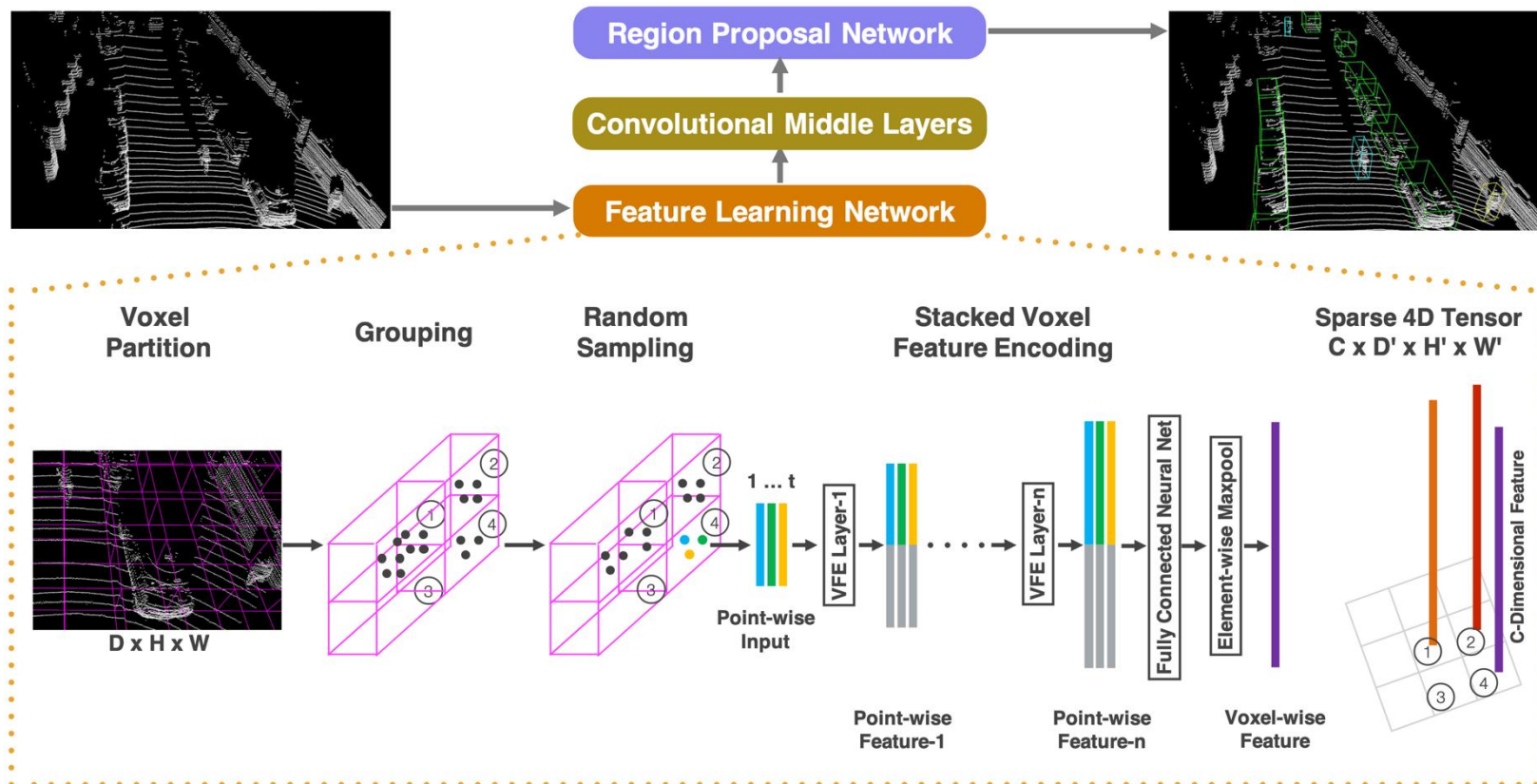
5) 3D Convolution (3D kernel)을 적용 → voxel-wise feature aggregate, progressively expand receptive field, add more context to shape description



VOXELNET

6) RetinaNet, YOLO 등 one-stage detection pipeline 적용 (classification + regression)

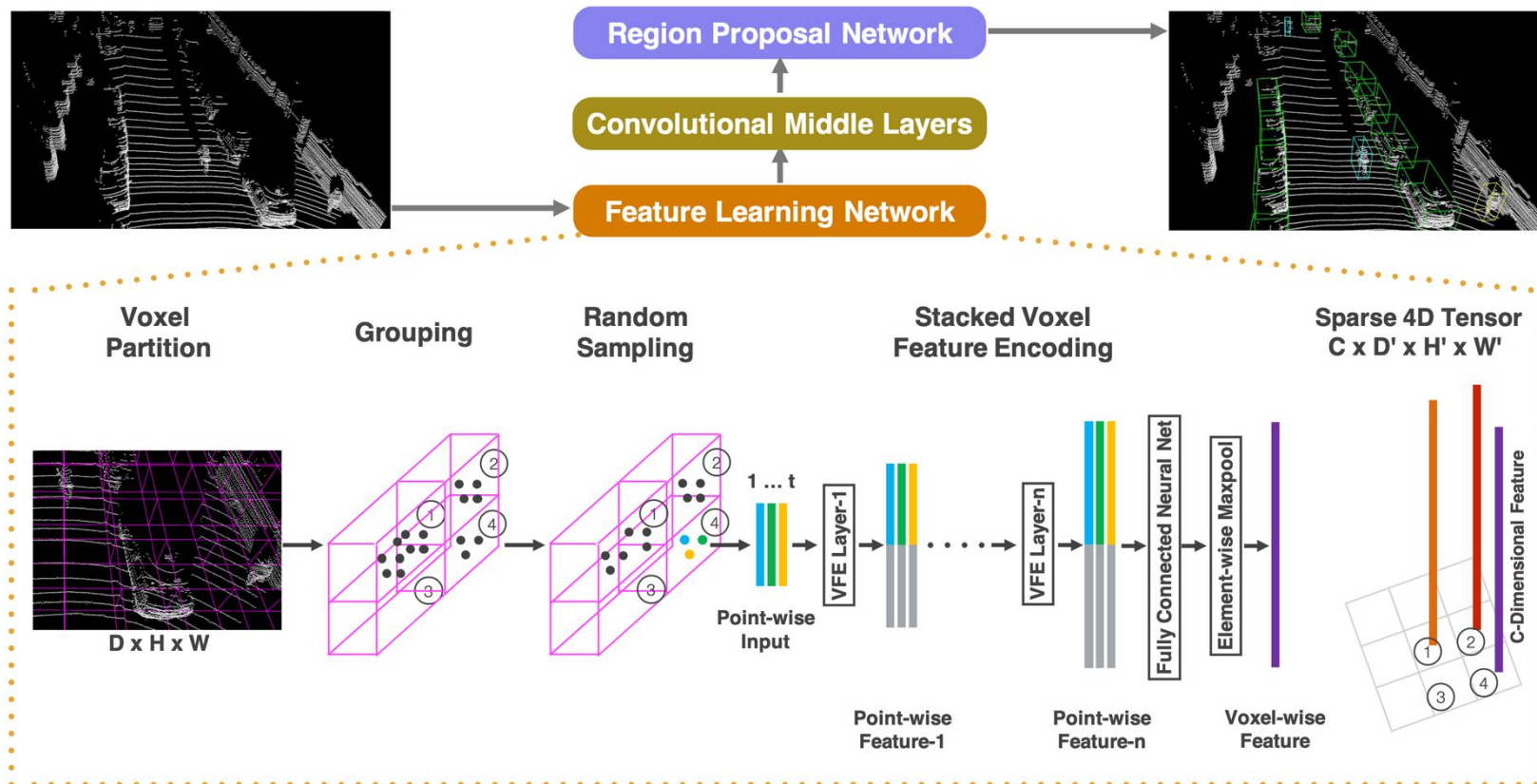
cf) 이름이 RPN이지만 two-stage 아니고 one-stage임



VOXELNET

단점 : 3D Convolution은 3D Kernel을 3D Space에서 slide하며 계산해야 해서 연산량  

➔ One-stage pipeline이지만 여전히 inference 느림



PIXOR

PIXOR = “OR”iented 3D object detection from “PIX”el-wise
neural network predictions

“Real-time” inference → How?

PIXOR: Real-time 3D Object Detection from Point Clouds

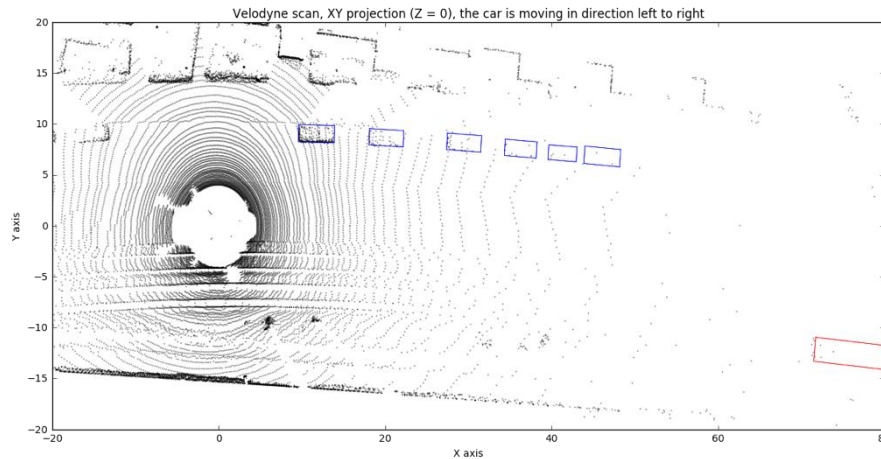
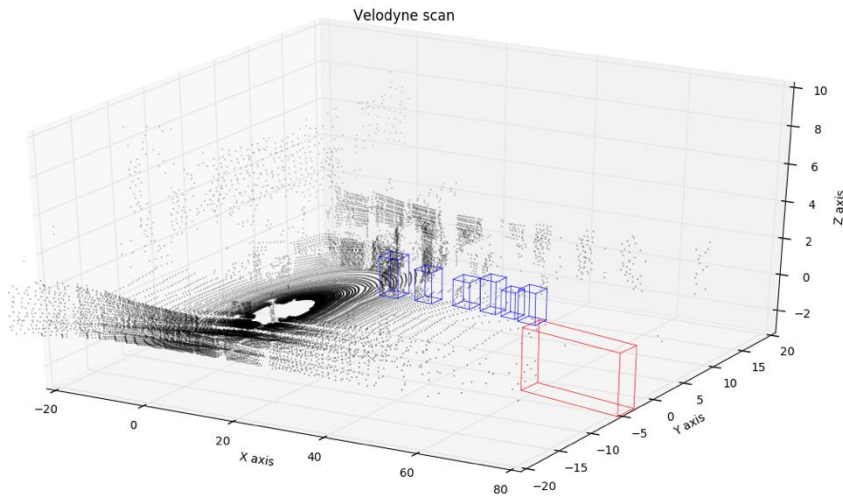
Bin Yang Wenjie Luo Raquel Urtasun
Uber Advanced Technologies Group University of Toronto
{byang10, wenjie, urtasun}@uber.com



PIXOR

LiDAR point cloud에 대한 Bird's Eye View (BEV) projection을 입력으로 사용

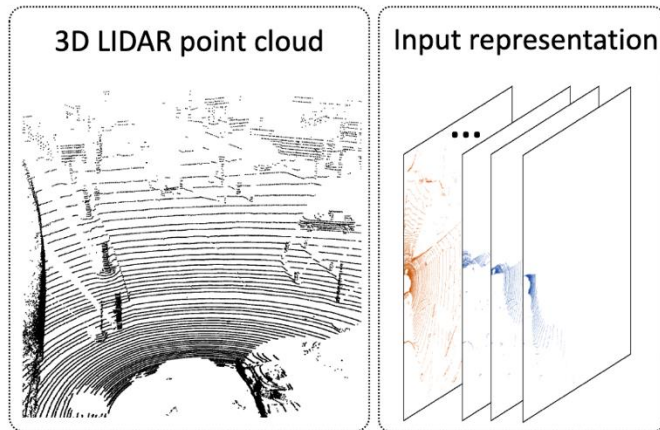
- 3D voxel grid와 비교해서 computationally efficient
- Projection 전후로 object의 physical size가 유지됨
- Object 3D bounding box들이 서로 겹치지 않음



PIXOR

1) 3D space의 x,y,z 축을 각각 0.1m 단위로 나누고 occupancy voxel로 변환
(voxel 내 point가 있으면 1, 없으면 0)

➔ z축을 2D Image의 channel 차원처럼 생각할 수 있음

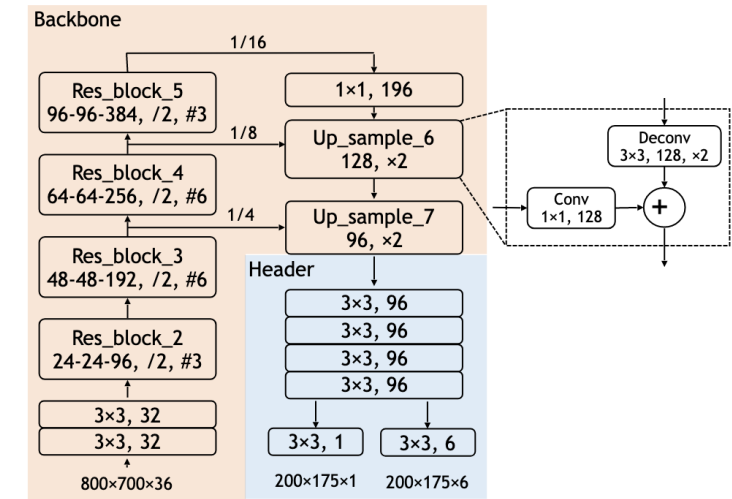
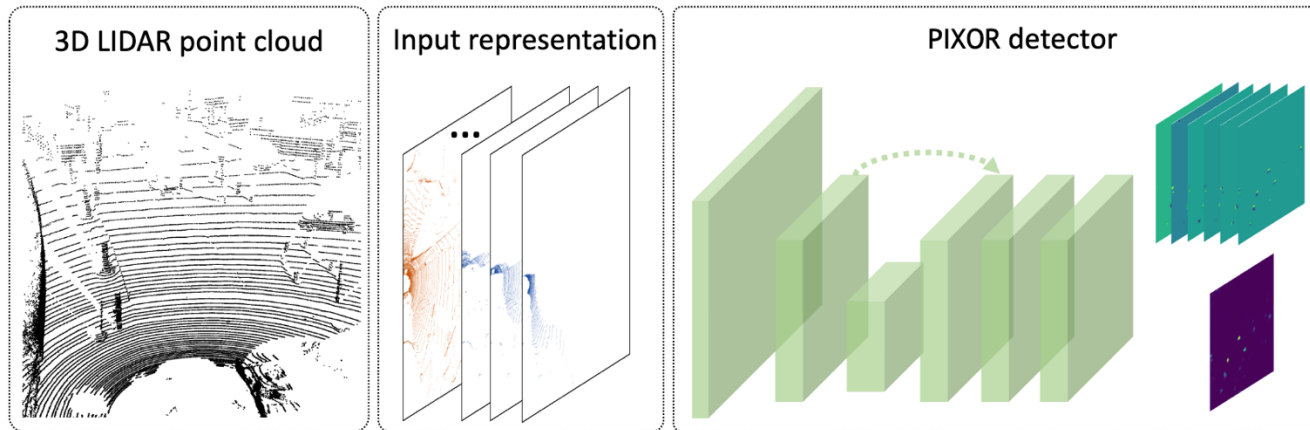


PIXOR

2) BEV representation에 2D Convolution을 적용

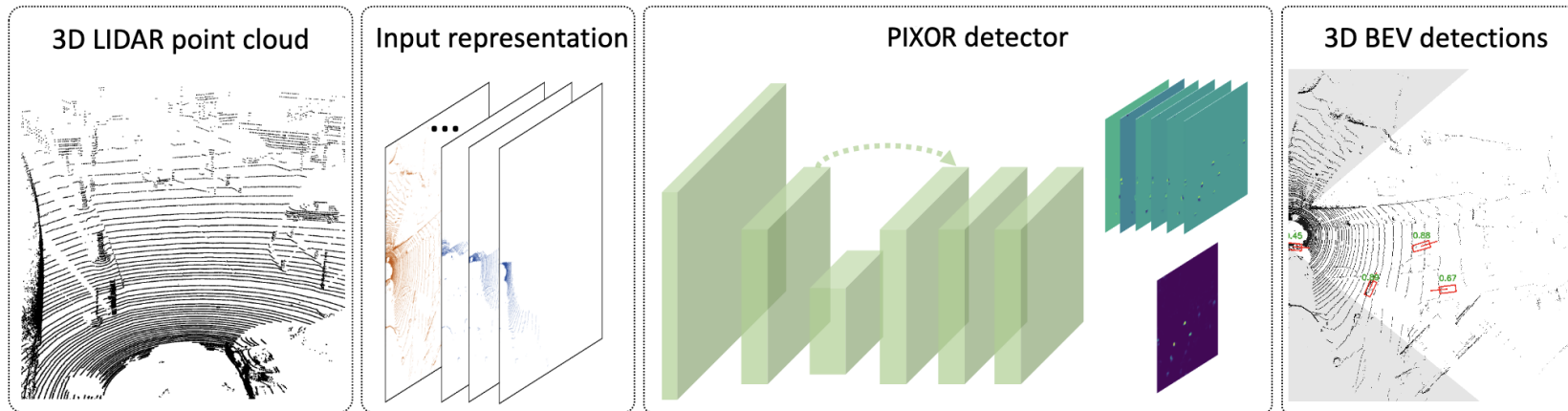
One-stage detector pipeline 적용 (classification + regression)

cf) RetinaNet + FPN 과 유사



PIXOR

3) Real-time inference (> 28 fps)



An abstract graphic on a light gray background. Two thin, dark gray lines intersect. One line is nearly vertical, starting from the top center and extending towards the bottom right. The other line is nearly horizontal, starting from the top left and extending towards the middle right. The intersection point is located to the left of the word 'SUMMARY'.

SUMMARY

SUMMARY

- 1. LiDAR Data Type & Backbone**
- 2. LiDAR-based 3D Object Detection**



NEXT LECTURE

**Multi-modal (Image + LiDAR) 3D Object detection
& Perception dataset**



Two thin, dark gray lines intersect diagonally on a light gray background. One line slopes downwards from left to right, and the other slopes upwards from left to right, crossing it at approximately one-third of the way across the frame.

QUESTIONS?